

VoxTree Voice Cloning Architecture

This document outlines the complete technical process of how voice cloning works within the VoxTree platform. VoxTree utilizes a **Hybrid Voice Architecture** to support both highly expressive storytelling (Text-to-Speech) and musical singing (Voice-to-Voice).

1. The Two Modes of Voice Cloning

VoxTree handles two distinct types of audio generation, each requiring a different AI model architecture:

- Text-to-Speech (TTS) for Storytelling:** Used when a family member is reading a book or narrating an educational video. We use **ElevenLabs** for this because it requires very little audio data (30–60 seconds) and produces highly expressive, emotionally intelligent speech from raw text.
- Voice-to-Voice (V2V) for Singing:** Used when a family member needs to sing a lullaby or an educational song. We use **RVC (Retrieval-based Voice Conversion)** for this because singing requires perfect preservation of pitch, melody, and vibrato from an original studio track. RVC requires 5 to 10 minutes of clean training audio.

2. Standard Voice Creation Workflow (ElevenLabs / TTS)

When a user creates a standard "Reading Voice" profile, the following data flow occurs:

- User Audio Capture:** The user records a 30-second audio snippet directly in the browser or uploads a pre-recorded file.
- Secure Upload:** The VoxTree backend generates an AWS S3 Pre-signed URL. The audio file is uploaded directly and securely from the user's browser to our private S3 bucket.
- Database Entry:** A record is created in the `family_voices` Supabase table with a status of `processing`.
- ElevenLabs API Submission:** The backend securely fetches the audio from S3 and transmits it to the ElevenLabs "Add Voice" API endpoint using our secure API key.
- Model Creation:** ElevenLabs processes the audio, extracts the unique vocal characteristics (timbre, tone, accent), and returns a unique `elevenlabs_voice_id`.
- Finalization:** The VoxTree database is updated with the `elevenlabs_voice_id`, and the voice profile is marked as `ready` for use.

3. Singing Voice Creation Workflow (RVC / V2V)

When a user wants to upgrade their voice profile to a "Singing Model", the process is more intensive:

- Dataset Upload:** The user navigates to the "Singing" tab in the Clones dashboard and uploads 5 to 10 minutes of clean, isolated speech.
- GPU Worker Hand-off:** Because RVC requires heavy GPU processing, the dataset is sent to an external GPU inference provider (e.g., Replicate or RunPod).
- Model Training:** The GPU worker trains a custom RVC model on the user's dataset. This process typically takes 15 to 30 minutes.
- Webhook Completion:** Once training is complete, the GPU worker sends a webhook back to VoxTree, updating the `rvc_model_id` and marking the `rvc_training_status` as `ready` in the database.

4. Clip Generation Process

When a user selects a Story or Video from the library and clicks "Generate", the system intelligently routes the request based on the content type:

Path A: The Content is a "Story" (TTS)

- The system reads the `text_script` attached to the content library item.
- The `text_script` and the user's `elevenlabs_voice_id` are sent to the ElevenLabs Text-to-Speech API.
- ElevenLabs returns the generated raw narration audio.
- An FFmpeg background worker mixes the generated narration with the story's `instrumental_url` (background music/sound effects) and syncs it to the video.

Path B: The Content is a "Song" (V2V)

- The system reads the `isolated_vocals_url` (an acapella studio track) attached to the content library item.
- The `isolated_vocals_url` and the user's `rvc_model_id` are sent to the external RVC GPU worker.
- The RVC model uses the original vocal track as a guide, swapping the original singer's voice with the user's cloned voice while keeping the exact same pitch and timing.
- An FFmpeg background worker mixes the new vocals with the song's `instrumental_url`.

Delivery

In both paths, the final rendered video/audio file is uploaded to S3, the `generated_clips` database record is marked as `ready`, and the media is streamed back to the user's dashboard!